

fftAnalysis.ts

frontend/signal-processing | TypeScript

```
/**
 * fftAnalysis.ts
 *
 * Frequency-domain analysis of the rPPG signal.
 *
 * While BioSync primarily uses time-domain variance for stress detection
 * (faster, more robust to short windows), this module provides an FFT-based
 * heart rate estimate for the signal confidence display and calibration.
 *
 * The dominant frequency in the rPPG signal corresponds to the user's
 * heart rate. A clean, periodic signal in the 60-100 BPM range (1-1.67 Hz)
 * indicates good signal quality. Noise or artefacts spread energy across
 * the spectrum, lowering the peak-to-noise ratio.
 *
 * Algorithm: Goertzel algorithm (efficient single-frequency DFT)
 * for each target BPM, avoiding a full FFT for performance.
 */

// ■■■ Types ■■■
export interface HeartRateEstimate {
  /** Estimated BPM (0 if signal insufficient) */
  bpm: number;
  /** Confidence in the estimate (0-1) – peak-to-mean ratio of frequency spectrum */
  confidence: number;
  /** Whether the estimated BPM is in a physiologically plausible range */
  plausible: boolean;
}

// ■■■ Constants ■■■

/** Typical resting/active heart rate range in BPM */
const HR_MIN_BPM = 45;
const HR_MAX_BPM = 150;

/** Sample rate: one rPPG sample per video frame at 30fps */
const SAMPLE_RATE_HZ = 30;

/** Minimum signal length for reliable frequency analysis */
const MIN_SAMPLES = 90; // 3 seconds at 30fps

// ■■■ Goertzel Algorithm ■■■

/**
 * Computes the power at a specific frequency using the Goertzel algorithm.
 * More efficient than FFT when only a few frequencies are of interest.
 *
 * @param signal - Array of time-domain samples
 * @param targetHz - Target frequency in Hz
 * @param sampleRate - Samples per second
 */
function goertzelPower(
  signal: number[],
  targetHz: number,
  sampleRate: number
): number {
  const N = signal.length;
  const omega = (2 * Math.PI * targetHz) / sampleRate;
  const coeff = 2 * Math.cos(omega);

  let s0 = 0, s1 = 0, s2 = 0;

  for (let i = 0; i < N; i++) {
    s0 = signal[i] + coeff * s1 - s2;
    s2 = s1;
    s1 = s0;
  }
}
```



```
if (hz >= 0.75 && hz <= 2.5) inBandPower += power;  
else outBandPower += power;  
}  
  
if (outBandPower === 0) return 10;  
return Math.min(10, (inBandPower / outBandPower) * 2);  
}
```